Purpose

PLAN-0 — planning/ Purpose + boundaries

Piece A — Purpose

File: `Axion/libraries/planning/PLAN-0_purpose.md`

---
library: planning
id: PLAN-0
schema_version: 1.0.0
status: draft

PLAN-0 — planning/ Purpose + Boundaries

Purpose
`planning/` defines Axion's **work planning mechanics**:
- how work is broken down into tasks (WBS)
- how requirements map to acceptance criteria (acceptance map)
- how a build plan sequences work deterministically (build plan)
- what "coverage" means for a plan (completeness rules)

Planning is the bridge between the canonical spec/templates and execution/verification.

What it governs (in scope)
- Work Breakdown Structure (WBS) format
- Acceptance map format (requirements → artifacts/tests)
- Build plan format (sequence, dependencies, milestones)
- Sequencing policies (deterministic task ordering)
- Coverage rules (what must be covered for a run to be "complete")

What it does NOT govern (out of scope)
- Stage ordering and run manifest → `orchestration/`
- Gate DSL evaluation → `gates/`
- Risk classes/overrides → `policy/`
- Intake form schemas → `intake/`
- Canonical spec schema → `canonical/`
- Standards resolution → `standards/`
- Template registry/selection/render rules → `templates/`
- Proof ledger and verification commands → `verification/`

Consumers

- Plan generation stage (S6)
- Template fill stage (S7) (uses plan to know what to generate)
- Gate evaluation (S8) (uses plan coverage rules)
- Operator UI (shows task list, sequencing, coverage gaps)

Determinism requirements
- Plans are a pure function of:
  - CANONICAL_SPEC
  - TEMPLATE_SELECTION
  - STANDARDS_SNAPSHOT
  - planning policies (this library, pinned)
- Sequencing is deterministic and recorded.

Piece B — Boundary checklist

File: `Axion/libraries/planning/PLAN-0_boundary_checklist.md`

---
library: planning
id: PLAN-0a
schema_version: 1.0.0
status: draft

PLAN-0a — Boundary Checklist

Belongs in `planning/` if it answers:
- “How do we break work into tasks?”
- “How do we map requirements to acceptance criteria?”
- “How do we sequence tasks deterministically?”
- “What does plan coverage mean?”

Does NOT belong in `planning/` if it answers:
- “What templates exist?” (templates)
- “How do we evaluate gates?” (gates)
- “What is the canonical schema?” (canonical)

Work breakdown structure

PLAN-1 — Work breakdown structure (WBS) format

Piece A — WBS model (concept)

File: `Axion/libraries/planning/PLAN-1_wbs_model.md`

---
library: planning
id: PLAN-1
schema_version: 1.0.0
status: draft

PLAN-1 — Work Breakdown Structure (WBS) Model

Purpose
Define a deterministic task list structure that can be generated from the canonical spec and used by template fill/executors.

WBS structure
A WBS is a list of work items, each with:
- a stable work_item_id
- title
- type (design/implementation/test/docs/etc.)
- inputs (refs to canonical entities/artifacts)
- outputs (what artifacts/proofs are produced)
- dependencies (work_item_id refs)
- status (planned/in_progress/done)
- verification hooks (what proof is required)

Design constraints
- IDs are stable and deterministic (based on canonical keys).
- Dependency graph is acyclic.
- Work items are ordered deterministically for display and execution.

Piece B — WBS schema (JSON Schema)

File: `Axion/libraries/planning/schemas/work_breakdown.v1.schema.json`

```
{
  "$schema": "https://json-schema.org/draft/2020-12/schema",
```

```
"schema_version": "1.0.0",
"id": "work_breakdown.v1",
"type": "object",
"required": ["wbs_id", "run_id", "items", "generated_at"],
"properties": {
  "wbs_id": { "type": "string", "pattern": "^WBS-[A-Z0-9]{6,}$" },
  "run_id": { "type": "string" },
  "generated_at": { "type": "string", "format": "date-time" },

  "items": {
    "type": "array",
    "minItems": 1,
    "items": { "$ref": "#/$defs/work_item" }
  }
},

"$defs": {
  "work_item": {
    "type": "object",
    "required": ["work_item_id", "title", "work_type", "inputs", "outputs", "depends_on", "status"],
    "properties": {
      "work_item_id": { "type": "string", "pattern": "^WI-[A-Z0-9]{6,}$" },
      "title": { "type": "string", "minLength": 3 },

      "work_type": {
        "type": "string",
        "enum": ["design", "implementation", "test", "docs", "data", "ops", "security"]
      },

      "inputs": {
        "type": "array",
        "items": { "type": "string" },
        "description": "Pointers to canonical entities, templates, or standards rules"
      },

      "outputs": {
        "type": "array",
        "items": { "type": "string" },
        "description": "Contract IDs or proof requirements"
      },

      "depends_on": { "type": "array", "items": { "type": "string" } },

      "status": { "type": "string", "enum": ["planned", "in_progress", "done"] },
```

```
      "priority": { "type": "string", "enum": ["low", "medium", "high"] }
    },
    "additionalProperties": false
  }
},

"additionalProperties": false
}
```

Piece C — Determinism rules (WBS)

File: `Axion/libraries/planning/PLAN-1_determinism_rules.md`

```
---
library: planning
id: PLAN-1a
schema_version: 1.0.0
status: draft
---

# PLAN-1a — Determinism Rules (WBS)

- work_item_id is generated deterministically from a canonical key (e.g., entity_id + work_type + output contract).
- items are ordered deterministically for storage:
  - priority (high → medium → low)
  - work_type (fixed order)
  - title
  - work_item_id
- depends_on must reference existing work_item_id values.
```

Piece D — Validation checklist

File: `Axion/libraries/planning/PLAN-1_validation_checklist.md`

```
---
library: planning
id: PLAN-1b
schema_version: 1.0.0
```

status: draft

PLAN-1b — Validation Checklist

- [ ] work_breakdown validates against schema
- [ ] work_item_id values are unique
- [ ] depends_on references are valid (strict mode)
- [ ] deterministic ordering applied
- [ ] outputs include expected contract/proof refs

Acceptance map format

PLAN-2 — Acceptance map format (requirements → acceptance criteria → proofs)

Piece A — Acceptance map model (concept)

File: `Axion/libraries/planning/PLAN-2_acceptance_map_model.md`

```
---
library: planning
id: PLAN-2
schema_version: 1.0.0
status: draft
---

# PLAN-2 — Acceptance Map Model

## Purpose
Define a structured map from requirements to:
- acceptance criteria
- artifacts that must be produced
- proofs/verification that must exist

This enables coverage checks in planning and verification gates.

## Structure
- requirement nodes (from canonical spec)
- acceptance criteria (testable statements)
- evidence requirements:
  - artifact contract IDs
  - proof ledger requirements
```

Piece B — Acceptance map schema (JSON Schema)

File: `Axion/libraries/planning/schemas/acceptance_map.v1.schema.json`

```
{
  "$schema": "https://json-schema.org/draft/2020-12/schema",
  "schema_version": "1.0.0",
  "id": "acceptance_map.v1",
  "type": "object",
```

```
"required": ["amap_id", "run_id", "requirements", "generated_at"],
"properties": {
  "amap_id": { "type": "string", "pattern": "^AMAP-[A-Z0-9]{6,}$" },
  "run_id": { "type": "string" },
  "generated_at": { "type": "string", "format": "date-time" },

  "requirements": {
    "type": "array",
    "minItems": 1,
    "items": { "$ref": "#/$defs/req" }
  }
},

"$defs": {
  "req": {
    "type": "object",
    "required": ["requirement_id", "title", "acceptance_criteria"],
    "properties": {
      "requirement_id": { "type": "string", "minLength": 2 },
      "title": { "type": "string", "minLength": 3 },

      "acceptance_criteria": {
        "type": "array",
        "minItems": 1,
        "items": { "$ref": "#/$defs/ac" }
      }
    },
    "additionalProperties": false
  },

  "ac": {
    "type": "object",
    "required": ["ac_id", "statement", "evidence"],
    "properties": {
      "ac_id": { "type": "string", "pattern": "^AC-[A-Z0-9]{6,}$" },
      "statement": { "type": "string", "minLength": 5 },

      "evidence": {
        "type": "object",
        "properties": {
          "artifact_contract_ids": { "type": "array", "items": { "type": "string" } },
          "proof_requirements": { "type": "array", "items": { "type": "string" } }
        },
        "additionalProperties": false
```

```
      }
    },
    "additionalProperties": false
  }
},

"additionalProperties": false
}
```

Piece C — Determinism rules (acceptance IDs + ordering)

File: `Axion/libraries/planning/PLAN-2_determinism_rules.md`

```
---
library: planning
id: PLAN-2a
schema_version: 1.0.0
status: draft
---

# PLAN-2a — Determinism Rules (Acceptance Map)

- requirement_id should reference canonical entities of type "requirement" (strict mode).
- ac_id is deterministic from:
  requirement_id + statement slug + evidence keys (hash-based).
- requirements are ordered deterministically:
  - requirement_id
- acceptance_criteria ordered deterministically:
  - ac_id
- evidence lists are sorted lexicographically.
```

Piece D — Validation checklist

File: `Axion/libraries/planning/PLAN-2_validation_checklist.md`

```
---
library: planning
id: PLAN-2b
schema_version: 1.0.0
status: draft
```

PLAN-2b — Validation Checklist

- [ ] acceptance_map validates against schema
- [ ] requirement_id values are present and stable
- [ ] ac_id values are unique across the map
- [ ] evidence fields use expected formats (contract IDs, proof requirements)
- [ ] deterministic ordering applied (requirements + ACs + evidence lists)

Build plan format + sequencing policies

PLAN-3 — Build plan format + sequencing policies

Piece A — Build plan model (concept)

File: `Axion/libraries/planning/PLAN-3_build_plan_model.md`

```
---
library: planning
id: PLAN-3
schema_version: 1.0.0
status: draft
---

# PLAN-3 — Build Plan Model

## Purpose
Define a deterministic execution plan that sequences work items (WBS) into phases/milestones.

## Build plan structure
- plan_id
- run_id
- phases[] (ordered)
- each phase contains ordered work_item_ids
- dependency rules enforce that prerequisites appear earlier
- optional milestones (gates, checkpoints)

## Why build_plan exists (vs WBS)
- WBS is the inventory of tasks
- Build plan is the ordering/packaging of tasks into an execution sequence
```

Piece B — Build plan schema (JSON Schema)

File: `Axion/libraries/planning/schemas/build_plan.v1.schema.json`

```
{
  "$schema": "https://json-schema.org/draft/2020-12/schema",
  "schema_version": "1.0.0",
  "id": "build_plan.v1",
  "type": "object",
  "required": ["plan_id", "run_id", "phases", "generated_at"],
  "properties": {
```

```
  "plan_id": { "type": "string", "pattern": "^PLAN-[A-Z0-9]{6,}$" },
  "run_id": { "type": "string" },
  "generated_at": { "type": "string", "format": "date-time" },

  "phases": {
    "type": "array",
    "minItems": 1,
    "items": { "$ref": "#/$defs/phase" }
  }
},

"$defs": {
  "phase": {
    "type": "object",
    "required": ["phase_id", "title", "order", "work_items"],
    "properties": {
      "phase_id": { "type": "string", "pattern": "^PHASE-[A-Z0-9]{4,}$" },
      "title": { "type": "string", "minLength": 3 },
      "order": { "type": "integer", "minimum": 1 },

      "work_items": {
        "type": "array",
        "minItems": 1,
        "items": { "type": "string" }
      },

      "milestones": {
        "type": "array",
        "items": {
          "type": "object",
          "required": ["milestone_id", "kind", "ref"],
          "properties": {
            "milestone_id": { "type": "string", "pattern": "^MS-[A-Z0-9]{4,}$" },
            "kind": { "type": "string", "enum": ["gate", "checkpoint", "release"] },
            "ref": { "type": "string" }
          },
          "additionalProperties": false
        }
      }
    },
    "additionalProperties": false
  }
},
```

```
  "additionalProperties": false
}
```

Piece C — Sequencing policy (deterministic ordering rules)

File: `Axion/libraries/planning/PLAN-3_sequencing_policies.md`

```
---
library: planning
id: PLAN-3a
schema_version: 1.0.0
status: draft
---

# PLAN-3a — Sequencing Policies (Deterministic)

## Goals
- dependencies first
- stable ordering for equal-priority items
- phases reflect typical build flow

## Phase model (default)
1) Foundation (repo, scaffolding, baseline config)
2) Core domain + data model
3) API/backend endpoints
4) Frontend/UI
5) Integrations
6) Testing + verification
7) Ops + release readiness

## Ordering inside a phase
Sort work items by:
1) dependency depth (lower first)
2) priority (high → low)
3) work_type fixed order:
   design → implementation → test → docs → security → ops → data
4) work_item_id lexicographic

## Dependency rule
- if A depends_on B, then B must appear:
  - in an earlier phase OR earlier in the same phase
- cycles are invalid and must fail planning gate
```

Piece D — Determinism rules

File: `Axion/libraries/planning/PLAN-3_determinism_rules.md`

```
---
library: planning
id: PLAN-3b
schema_version: 1.0.0
status: draft
---

# PLAN-3b — Determinism Rules (Build Plan)

- phases are ordered by phase.order (unique integers).
- work_items order is deterministic per PLAN-3a.
- all work_item_ids referenced must exist in WBS.
- dependency constraints must be satisfied.
```

Piece E — Validation checklist

File: `Axion/libraries/planning/PLAN-3_validation_checklist.md`

```
---
library: planning
id: PLAN-3c
schema_version: 1.0.0
status: draft
---

# PLAN-3c — Validation Checklist

- [ ] build_plan validates against schema
- [ ] phase.order values are unique and start at 1
- [ ] every referenced work_item_id exists in WBS (strict mode)
- [ ] dependency ordering constraints satisfied
- [ ] no dependency cycles exist
- [ ] deterministic ordering applied inside phases
```

Coverage rules

PLAN-4 — Coverage rules (what must be covered for a run to be “complete”)

Piece A — Coverage model (concept)

File: `Axion/libraries/planning/PLAN-4_coverage_model.md`

```
---
library: planning
id: PLAN-4
schema_version: 1.0.0
status: draft
---

# PLAN-4 — Coverage Model

## Purpose
Define measurable coverage requirements so Axion can say:
- the plan is complete
- nothing critical was missed
- coverage gates can be evaluated deterministically

Coverage applies to:
- canonical entities (what exists must be addressed)
- selected templates (what was selected must be filled and validated)
- acceptance criteria (what must be proven)

## Coverage dimensions
1) **Entity coverage**
- each required canonical entity type must map to at least one work item

2) **Template coverage**
- each selected template must have a corresponding plan segment/work items

3) **Acceptance coverage**
- each requirement in acceptance map must have acceptance criteria + evidence requirements
```

Piece B — Coverage rules registry schema

File: `Axion/libraries/planning/schemas/plan_coverage_rules.v1.schema.json`

```
{
  "$schema": "https://json-schema.org/draft/2020-12/schema",
  "schema_version": "1.0.0",
  "id": "plan_coverage_rules.v1",
  "type": "object",
  "required": ["rules_id", "schema_version", "rules"],
  "properties": {
    "rules_id": { "type": "string", "minLength": 3 },
    "schema_version": { "type": "string", "minLength": 1 },
    "rules": {
      "type": "array",
      "minItems": 1,
      "items": { "$ref": "#/$defs/rule" }
    }
  },

  "$defs": {
    "rule": {
      "type": "object",
      "required": ["rule_id", "category", "requirement", "severity"],
      "properties": {
        "rule_id": { "type": "string", "pattern": "^COV-[A-Z0-9_\\-]+$" },
        "category": { "type": "string", "enum": ["entity", "template", "acceptance"] },
        "requirement": { "type": "string", "minLength": 3 },
        "severity": { "type": "string", "enum": ["must", "should"] }
      },
      "additionalProperties": false
    }
  },

  "additionalProperties": false
}
```

Piece C — Starter coverage rules registry

File: `Axion/libraries/planning/registries/plan_coverage_rules.v1.json`

```
{
  "schema_version": "1.0.0",
  "rules_id": "plan_coverage_rules.v1",
  "rules": [
```

```
    { "rule_id": "COV-ENTITY-COMPONENTS", "category": "entity", "requirement": "Every component entity must map to >=1 work item.", "severity": "must" },
    { "rule_id": "COV-ENTITY-ENDPOINTS", "category": "entity", "requirement": "Every endpoint entity must map to >=1 work item.", "severity": "must" },
    { "rule_id": "COV-TEMPLATES-ALL", "category": "template", "requirement": "Every selected template must map to >=1 work item.", "severity": "must" },
    { "rule_id": "COV-ACCEPTANCE-ALL", "category": "acceptance", "requirement": "Every requirement must have >=1 acceptance criterion with evidence.", "severity": "must" }
  ]
}
```

Piece D — Coverage evaluation output (optional artifact)

File:
`Axion/libraries/planning/schemas/plan_coverage_report.v1.schema.json`

```
{
  "$schema": "https://json-schema.org/draft/2020-12/schema",
  "schema_version": "1.0.0",
  "id": "plan_coverage_report.v1",
  "type": "object",
  "required": ["run_id", "status", "results", "evaluated_at"],
  "properties": {
    "run_id": { "type": "string" },
    "status": { "type": "string", "enum": ["pass", "fail", "warn"] },
    "results": {
      "type": "array",
      "items": {
        "type": "object",
        "required": ["rule_id", "status", "details"],
        "properties": {
          "rule_id": { "type": "string" },
          "status": { "type": "string", "enum": ["pass", "fail", "warn"] },
          "details": { "type": "object", "additionalProperties": true }
        },
        "additionalProperties": false
      }
    },
    "evaluated_at": { "type": "string", "format": "date-time" }
  },
  "additionalProperties": false
}
```

Piece E — Determinism rules (coverage)

File: `Axion/libraries/planning/PLAN-4_determinism_rules.md`

```
---
library: planning
id: PLAN-4a
schema_version: 1.0.0
status: draft
---

# PLAN-4a — Determinism Rules (Coverage)

- Coverage rules registry is pinned.
- Coverage evaluation uses only:
  - canonical_spec
  - template_selection
  - work_breakdown
  - acceptance_map
- Missing items lists are sorted deterministically (by entity_id/template_id/requirement_id).
```

Piece F — Validation checklist

File: `Axion/libraries/planning/PLAN-4_validation_checklist.md`

```
---
library: planning
id: PLAN-4b
schema_version: 1.0.0
status: draft
---

# PLAN-4b — Validation Checklist

- [ ] plan_coverage_rules validates against schema
- [ ] rules cover entity, template, and acceptance categories
- [ ] coverage evaluation (if used) validates against schema
- [ ] evaluation is deterministic and references pinned inputs only
```

Gates

PLAN-5 — Gates (plan coverage, acceptance mapping)

Piece A — Planning gates intent

File: `Axion/libraries/planning/PLAN-5_planning_gates.md`

```
---
library: planning
id: PLAN-5
schema_version: 1.0.0
status: draft
---

# PLAN-5 — Planning Gates

## Purpose
Ensure S6 plan generation produces a coherent, complete, deterministic plan that downstream stages can execute and verify.

## Gate set (minimum)

### PLAN-GATE-01 — WBS valid
- WORK_BREAKDOWN exists
- validates against work_breakdown.v1
- depends_on references valid work_item_ids

### PLAN-GATE-02 — Acceptance map valid
- ACCEPTANCE_MAP exists
- validates against acceptance_map.v1
- evidence requirements present for each acceptance criterion

### PLAN-GATE-03 — Build plan valid
- BUILD_PLAN exists
- validates against build_plan.v1
- phases ordered; references valid work_item_ids

### PLAN-GATE-04 — Dependency ordering valid (no cycles)
- dependency graph is acyclic
- build plan ordering respects depends_on constraints

### PLAN-GATE-05 — Coverage satisfied (risk-based)
- evaluate pinned coverage rules (PLAN-4)
```

- missing coverage fails under PROD/COMPLIANCE
- prototype may warn/pause (policy controlled)

PLAN-GATE-06 — Plan artifacts pinned in run manifest
- manifest includes WORK_BREAKDOWN, ACCEPTANCE_MAP, BUILD_PLAN artifacts

---

Piece B — Mapping to pipeline gate (G6_PLAN_COVERAGE)

File: `Axion/libraries/planning/PLAN-5_gate_mapping.md`

```
---
library: planning
id: PLAN-5a
schema_version: 1.0.0
status: draft
---

# PLAN-5a — Mapping to Pipeline Gate

Pipeline gate: **G6_PLAN_COVERAGE**

G6 passes if:
- PLAN-GATE-01..04 pass
- PLAN-GATE-05 passes (or is policy-approved when required)
- PLAN-GATE-06 passes

Default risk handling:
- PROTOTYPE: coverage gaps may warn/pause
- PROD: coverage gaps fail unless approved override
- COMPLIANCE: coverage gaps fail by default
```

Piece C — Machine-readable gate spec (optional)

File: `Axion/libraries/planning/PLAN-5_planning_gates.spec.json`

```
{
  "schema_version": "1.0.0",
  "contract_id": "PLAN-5b",
  "gates": [
```

```
    { "gate_id": "PLAN-GATE-01", "name": "WBS valid", "severity": "hard_stop", "checks": ["WORK_BREAKDOWN exists", "schema_valid(work_breakdown.v1)"] },
    { "gate_id": "PLAN-GATE-02", "name": "Acceptance map valid", "severity": "hard_stop", "checks": ["ACCEPTANCE_MAP exists", "schema_valid(acceptance_map.v1)", "evidence present"] },
    { "gate_id": "PLAN-GATE-03", "name": "Build plan valid", "severity": "hard_stop", "checks": ["BUILD_PLAN exists", "schema_valid(build_plan.v1)"] },
    { "gate_id": "PLAN-GATE-04", "name": "Dependency ordering valid", "severity": "hard_stop", "checks": ["no cycles", "plan respects depends_on ordering"] },
    { "gate_id": "PLAN-GATE-05", "name": "Coverage satisfied", "severity": "policy_controlled", "checks": ["coverage rules pass"] },
    { "gate_id": "PLAN-GATE-06", "name": "Plan artifacts pinned in manifest", "severity": "hard_stop", "checks": ["manifest references WBS/AMAP/PLAN artifacts"] }
  ]
}
```

Piece D — Evidence requirements

File: `Axion/libraries/planning/PLAN-5_evidence_requirements.md`

```
---
library: planning
id: PLAN-5c
schema_version: 1.0.0
status: draft
---

# PLAN-5c — Evidence Requirements (Planning Failures)

On failure, evidence must include:
- pointers to the failing artifact (WBS/AMAP/PLAN)
- missing work_item_ids or invalid depends_on refs
- cycle details (list of involved work_item_ids)
- coverage gaps:
  - missing entity_ids
  - missing template_ids
  - missing requirement_ids
- remediation:
  - add missing work items
  - add acceptance criteria/evidence requirements
  - fix dependency cycles
  - rerun S6 and downstream stages
```

Piece E — Determinism rules

File: `Axion/libraries/planning/PLAN-5_determinism_rules.md`

```
---
library: planning
id: PLAN-5d
schema_version: 1.0.0
status: draft
---

# PLAN-5d — Determinism Rules (Planning Gates)

- Gates run in fixed order PLAN-GATE-01..06.
- Coverage evaluation uses only pinned artifacts and pinned coverage rules.
- Missing lists are deterministically sorted.
- Policy-controlled outcomes must reference pinned policy decisions.
```

Piece F — Validation checklist

File: `Axion/libraries/planning/PLAN-5_validation_checklist.md`

```
---
library: planning
id: PLAN-5e
schema_version: 1.0.0
status: draft
---

# PLAN-5e — Validation Checklist

- [ ] Planning gate set covers WBS, acceptance map, build plan, dependency correctness, coverage, manifest pinning
- [ ] G6 mapping is explicit
- [ ] Coverage gating is risk-class + policy controlled
- [ ] Evidence pointers + remediation included
```

Say **“next”** for **PLAN-6 Minimum viable planning set**.

Minimum viable planning

PLAN-6 — Minimum viable planning set (runtime-required files + “done”)

Piece A — MV planning inventory

File: `Axion/libraries/planning/PLAN-6_minimum_viable_set.md`

---
library: planning
id: PLAN-6
schema_version: 1.0.0
status: draft

PLAN-6 — Minimum Viable planning/ Set

Goal
Define the smallest planning library set required to produce deterministic WBS, acceptance map, and build plan with enforceable coverage rules.

Required runtime files (minimum)
1) Core docs
- PLAN-0_purpose.md
- PLAN-0_boundary_checklist.md
- PLAN-1_wbs_model.md
- PLAN-1_determinism_rules.md
- PLAN-1_validation_checklist.md
- PLAN-2_acceptance_map_model.md
- PLAN-2_determinism_rules.md
- PLAN-2_validation_checklist.md
- PLAN-3_build_plan_model.md
- PLAN-3_sequencing_policies.md
- PLAN-3_determinism_rules.md
- PLAN-3_validation_checklist.md
- PLAN-4_coverage_model.md
- PLAN-4_determinism_rules.md
- PLAN-4_validation_checklist.md
- PLAN-5_planning_gates.md
- PLAN-5_gate_mapping.md
- PLAN-5_evidence_requirements.md
- PLAN-5_determinism_rules.md
- PLAN-5_validation_checklist.md

- PLAN-6_minimum_viable_set.md

2) Schemas (must be in schema_registry)
- schemas/work_breakdown.v1.schema.json
- schemas/acceptance_map.v1.schema.json
- schemas/build_plan.v1.schema.json
- schemas/plan_coverage_rules.v1.schema.json
- schemas/plan_coverage_report.v1.schema.json (optional)

3) Registries (runtime defaults)
- registries/plan_coverage_rules.v1.json

4) Gate specs (optional)
- PLAN-5_planning_gates.spec.json

Piece B — Definition of done (planning/)

File: `Axion/libraries/planning/PLAN-6_definition_of_done.md`

---
library: planning
id: PLAN-6a
schema_version: 1.0.0
status: draft

PLAN-6a — planning/ Definition of Done

planning/ is “done” when:

Schemas + registries
- [ ] All planning schemas validate (JSON Schema check)
- [ ] coverage rules registry validates and is pinned

Runtime behavior (contract-level)
- [ ] S6 produces WORK_BREAKDOWN, ACCEPTANCE_MAP, BUILD_PLAN deterministically
- [ ] Work items have stable IDs and valid dependency references
- [ ] Build plan respects dependencies and has no cycles
- [ ] Acceptance map includes evidence requirements per acceptance criterion
- [ ] Coverage can be evaluated deterministically from pinned artifacts

Gates

- [ ] PLAN-GATE-01..06 implemented and mapped to G6_PLAN_COVERAGE
- [ ] Coverage gaps handled per risk class + policy decisions
- [ ] Plan artifacts pinned in run manifest

Piece C — Minimal folder tree

File: `Axion/libraries/planning/PLAN-6_minimal_tree.txt`

```
Axion/libraries/planning/
  PLAN-0_purpose.md
  PLAN-0_boundary_checklist.md
  PLAN-1_wbs_model.md
  PLAN-1_determinism_rules.md
  PLAN-1_validation_checklist.md
  PLAN-2_acceptance_map_model.md
  PLAN-2_determinism_rules.md
  PLAN-2_validation_checklist.md
  PLAN-3_build_plan_model.md
  PLAN-3_sequencing_policies.md
  PLAN-3_determinism_rules.md
  PLAN-3_validation_checklist.md
  PLAN-4_coverage_model.md
  PLAN-4_determinism_rules.md
  PLAN-4_validation_checklist.md
  PLAN-5_planning_gates.md
  PLAN-5_planning_gates.spec.json
  PLAN-5_gate_mapping.md
  PLAN-5_evidence_requirements.md
  PLAN-5_determinism_rules.md
  PLAN-5_validation_checklist.md
  PLAN-6_minimum_viable_set.md
  PLAN-6_definition_of_done.md
  PLAN-6_minimal_tree.txt

  schemas/
    work_breakdown.v1.schema.json
    acceptance_map.v1.schema.json
    build_plan.v1.schema.json
    plan_coverage_rules.v1.schema.json
    plan_coverage_report.v1.schema.json

  registries/
```

plan_coverage_rules.v1.json

Say **“next”** to start the next library folder. Recommended next is **verification/** (VER-0).